RUNBOOK — From the exam room to submission

This is **your direction for the whole exam**. Follow it top to bottom. Every step says **where** you do it, **what** to do, and **why**, in plain English, then gives you the exact prompt or command.

Where things happen, shown at the start of each step:

- **[TERMINAL]**: the black command window on the lab PC
- **[QODER: Ask]**: Qoder chat, Ask mode (it can only talk, it can't change files)
- **[QODER: Agent]**: Qoder chat, Agent mode (it edits files and runs commands)
- **[BROWSER]**: Chrome/Firefox at http://localhost:3000
- **[TEXT EDITOR]**: any plain text editor on the PC (Qoder itself works)

Words in [square brackets] inside prompts are things you replace before sending.

Other docs this one points to (all in your `pack` folder):

- `STACK-A-NEXTJS.md`: setup details, reference database code, Node 18 fixes
- `PLAN-PROMPT.md`, `DESIGN.md`, `PROMPTS.md`: where the prompts below come from (copied here in full, so you don't need to open them)
- `COMMITS.md`: git rescue commands
- `DATABASES-AND-DOCKER.md`: SQL cheat sheet

The 30-second version

1. Get the files, set up git, sign in to Qoder.
2. **08:00:** plan with the Master Plan Prompt (no coding).
3. **08:30:** create the project, copy in the files, first push.
4. Check Qoder reads your rules.
5. Build the **database** first, and test it.
6. Build the look (design), then features **one task per chat**.
7. After every task: check it in the browser, then commit. Push every hour.
8. **16:00:** freeze. Clean up, final checks, push, submit.

How the app works: why the database part is simpler than Supabase

With Supabase/Postgres you had a separate database **server**, an account, API keys, a connection string, network calls and permissions. That's where the complexity came from.

Here you use **SQLite**. The whole database is **one file** (`data/app.db`) inside your project. There's no server, no login and no keys. Your code opens the file directly.

Every feature follows the same path:

```
Page in browser → API route (src/app/api/.../route.ts) → lib function (src/lib/tasks.ts) → db.ts
→ data/app.db
   (what you see) (receives the request) (the SQL lives here) (opens the file) (the data)
```

- `src/lib/migrations.ts`: the list of `CREATE TABLE` statements. It runs automatically when the app starts. It is the **only** place tables are created or changed.
- `src/lib/db.ts`: opens the database file once and runs the migrations. You never touch it after setup.
- `src/lib/<thing>.ts`: one file per kind of data (tasks, projects…) with functions like `createTask`, `listTasks`, `archiveTask`. **All SQL lives here.**
- **API routes and pages** never write SQL. They just call the lib functions.

- **Tests** use a throwaway in-memory database (`DB_PATH=:memory:`), so they never mess up your real data.

If something about data breaks, it's almost always in one of three places: the **migration** (table shape), the **lib function** (the SQL), or the **page not refreshing** (it shows old data). The database prompts below cover all three.

STEP 0 — Arrive (07:30–07:45)

Why: setup problems (logins, git, Qoder sign-in) eat time. Get them out of the way before the clock starts.

- Phone with: this runbook (PDF), your **GitHub token** (`ghp_...`), your Qoder login.
- Earphones, water, snacks.
- Ask the invigilator: *"Are we allowed to log in and set up git before 08:00?"* If not, do Step 1 at 08:00. It's setup, not coding.

STEP 1 — Set up the PC (before 08:00, ~10 min)

1a. [TERMINAL] Check the machine

Why: confirms it's the same setup as the test (Node 18). The pinned versions in Step 4 depend on it.

```
node -v
npm -v
git --version
python3 --version
```

Expect `v18.19.1` for node. Anything 18 or higher is fine.

1b. [TERMINAL] Get your files

Why: your rulebook, design and prompts need to be on the PC.

```
cd ~
git clone https://github.com/Suhai1/coms3011a-test-practice.git pack
ls pack
```

You should see `AGENTS.md`, `DESIGN.md`, `RUNBOOK.md` and the rest. **If GitHub is blocked:** download them from the Moodle upload. **Last resort:** read this runbook on your phone.

1c. [TERMINAL] Tell git who you are

Why: git needs your name for commits, and the cache remembers your token for the day, so you only paste it once.

```
git config --global user.name "Suhail Seedat"
git config --global user.email "[your GitHub email]"
git config --global credential.helper 'cache --timeout=36000'
```

1d. Open Qoder and sign in

Don't open a folder yet. Find the **mode** switch (Ask / Agent) and the **model** switch (Auto / Ultimate / Fast / Efficient) at the bottom of the chat box (**Ctrl+L** opens chat).

STEP 2 — Planning (08:00–08:30, NO coding)

2a. Breathe for 2 minutes

Why: you have 7.5 hours. Two calm minutes make the next 450 better.

2b. Read the brief twice

Why: marks come from the brief's exact wording. Note every "must", "required", "done when", and anything about **marks**.

2c. [TEXT EDITOR] Save the brief as a file

Why: you'll give it to Qoder all day as `@BRIEF.md`.

Copy the entire brief text into a new file and save it as `~/BRIEF.md`. Copying text isn't coding.

2d. [QODER: Ask, model Ultimate] Run the Master Plan Prompt

Why: this turns the brief into a numbered task list with database tables, marks and a "done when" check per task, which is the same thing you did in the test, but more structured. Ultimate is the smartest model, and the plan is worth it.

New chat → **Ask** → **Ultimate.** Paste this, then paste the **whole brief** after the last line:

```
You are my senior engineer and exam strategist. I have a 7.5-hour practical software exam,
building an application with you, an AI coding agent. It is marked LIVE by a demonstrator at the
end, against the brief's requirements and "done when" conditions. Your job right now is ONLY to
plan. Do NOT write application code, and do not create or edit files.

## My environment (fixed: plan around it)
- Ubuntu lab PC, Node 18.19.1, firewalled internet (npm registry and docs are reachable; assume
CDNs, Google Fonts and most other sites are blocked).
- Stack: Next.js 15.5.27 (App Router) + TypeScript (strict) + Tailwind 4.1.18 (pinned) + SQLite
via better-sqlite3 11.10.0 (fallback: sql.js), Vitest 3.2.4 for tests. Raw SQL with ?
placeholders, no ORM.
- Every package must support Node 18. No external services, no auth providers, no CDN scripts.
- Business logic lives in src/lib/<resource>.ts; API routes and pages are thin and call lib
functions.
- My full rules are in AGENTS.md (I'll attach it during the build). Key ones: never delete rows
(archive with archived_at), never store derived values, fixed enums enforced with CHECK
constraints, versioned migrations (never drop/recreate), index filtered and sorted columns, no N+1
queries, cycle guards on any graph/tree, tests against an in-memory database (never mocked).

## How I will work
- One task per fresh chat. Each task must be finished, verified in the browser, tested and
committed before the next starts.
- I will paste a task prompt like: "@PLAN.md @AGENTS.md Do task N only."
- So every task must be self-contained, small (10–25 minutes of agent work), and checkable by me
in under 60 seconds.

## Produce the plan in EXACTLY this structure (markdown):

### 1. The brief in one paragraph
What the app is, who uses it, and how it will be marked.

### 2. Marks map
A table: requirement / feature | marks or weight (quote the brief) | required or optional | "done
when" condition quoted word-for-word from the brief. If the brief gives no marks, rank by
importance.

### 3. Hard rules and traps
Every constraint in the brief that an AI agent is likely to break by accident (e.g. deletion,
stored derived values, fixed lists, time zones, persistence across restart). One line each,
quoting the brief.

### 4. Data model
Complete SQLite CREATE TABLE and CREATE INDEX statements for migration 1, with a one-line reason
for each table. Include archived_at, created_at, updated_at, CHECK constraints for fixed enums,
foreign keys, and join tables for many-to-many.

### 5. Screens and API
- Pages/views: route, what's on it, which tasks build it.
- API routes: method + path, input, output, error cases.

### 6. Task list (the most important part)
Numbered tasks in build order. Rules:
- Task 1 = foundation: db.ts, migrations, design tokens from DESIGN.md, layout/nav, one passing
test. No features.
- Then ALL required/core features, one per task, smallest working version first.
- Then optional features in order of (marks ÷ effort), highest first.
- Insert a "VERIFY" task after every 3–4 build tasks: run all tests, click through the full user
journey, fix regressions only.
- Put a clear line "---- MINIMUM PASSING VERSION ENDS HERE ----" after the last core task.
- Final tasks: seed data with realistic content, design pass against DESIGN.md, README with run
instructions, final check.
```

```
For EACH task use this exact format:
**Task N — <name>** (est. <minutes> min, <marks> marks, depends on: <task numbers or none>)
- Goal: <one sentence, user-visible outcome>
- Files: <files to create or change>
- Rules to respect: <the specific AGENTS.md / brief rules this task could break>
- Done when: <a concrete check I can do in the browser or terminal in under 60 seconds, mirroring the brief's wording where possible>
- Tests: <the behaviour tests to add, including the awkward case>

### 7. Time budget
Map the tasks onto the day: 08:30 start, 1-hour lunch around 12:30–13:30 (assume this unless the brief says otherwise), feature freeze at 16:00, finished by 16:45. Mark where I should be by 10:30, 12:30 and 15:00. If the task list can't fit, say which optional tasks to drop.

### 8. Risks and ambiguities
- Anything unclear or contradictory in the brief, with the interpretation you recommend and why (I may ask the lecturer).
- Anything that may not work on Node 18 or behind the firewall, with a fallback.

Be specific to THIS brief. No generic advice. If something isn't in the brief, don't invent requirements; list it under ambiguities instead.

=== PASTE THE FULL BRIEF BELOW THIS LINE ===
```

Insert: the full brief text, right after `=== PASTE THE FULL BRIEF BELOW THIS LINE ===`.

2e. [QODER: Ask] Correct the plan

Why: the first plan is never perfect. Fixing it now costs 2 minutes; fixing it at 14:00 costs an hour.

Check that it has: a **marks map**, **hard rules**, **CREATE TABLE** statements, a **numbered task list** with "Done when" for each, and the **MINIMUM PASSING VERSION** line. Then push back, for example:

```
The brief says [quote]. Your plan has [what's wrong]. Fix that and reprint the whole plan.
```

```
Move Task [N] before Task [M], because [reason]. Reprint the whole plan.
```

```
Check your database tables against every requirement in the brief. List any requirement whose data has nowhere to be stored, then reprint the plan with fixed tables.
```

2f. [TEXT EDITOR] Save the plan

Why: chats can get lost when you open a folder. A file can't.

Copy the final plan (use the copy button on the message) into a new file and save it as `~/PLAN.md`.

Checkpoint at 08:30: you have `~/BRIEF.md` and `~/PLAN.md`, and you understand tasks 1–5.

STEP 3 — Create the project (08:30, ~15 min)

3a. [TERMINAL] Create it with the versions that work on Node 18

Why: the newest versions **break** on Node 18 (that's what happened in the test). These exact versions were tested on Node 18.19.1.

```
cd ~
npx -y create-next-app@15.5.27 exam-app --ts --tailwind --eslint --app --src-dir --use-npm --import-alias "@/*" --turbopack --yes
cd exam-app
npm install -E -D tailwindcss@4.1.18 @tailwindcss/postcss@4.1.18
npm install -E better-sqlite3@11.10.0
npm install -E -D @types/better-sqlite3 vitest@3.2.4 tsx
mkdir -p data src/lib src/tests scripts
npm pkg set scripts.test="DB_PATH=:memory: vitest run"
npm pkg set scripts.seed="tsx scripts/seed.ts"
npm run build
```

It worked if: `npm run build` ends with a list of routes and no red errors.

- Error mentioning **tailwindcss-oxide** → run the `tailwindcss@4.1.18` line again, then `npm run build`.
- **better-sqlite3** install fails (errors mentioning `prebuild`, `gyp` or GitHub) → carry on anyway; you'll use prompt **DB-FIX** in Step 5.

3b. [TERMINAL] Put the right files in the project folder

Why: Qoder can only see files **inside** the folder you open. These five files are its instructions.

```
cp ~/pack/AGENTS.md ~/pack/DESIGN.md ~/pack/COMMITS.md ~/BRIEF.md ~/PLAN.md ~/exam-app/
printf "data/*.db\ndata/*.db-*\n.env\n" >> .gitignore
ls
```

Your project folder should now contain:

File	What it's for
`AGENTS.md`	Rules Qoder follows **automatically** on every request (stack, Node 18, database rules)
`DESIGN.md`	How the app should look
`COMMITS.md`	Commit message style
`BRIEF.md`	The exam brief
`PLAN.md`	Your task list, which you'll tick off
`src/`, `package.json`, …	The Next.js app

Keep in `~/pack`, NOT in the project: `RUNBOOK.md`, `PROMPTS.md`, `PLAN-PROMPT.md`, the stack guides. They're for you, not Qoder, and they'd just clutter its context.

3c. [TEXT EDITOR] Fill in the bottom of AGENTS.md

Why: this gives Qoder the brief's specific rules on every single request.

Open `~/exam-app/AGENTS.md`, scroll to **"8. Project-specific notes"**, and replace the placeholders with: the app in one line, the **hard rules** from PLAN.md section 3, and the core features.

3d. [TERMINAL] First commit and push

Why: proves your token works **now**, not at 16:55. After this, you can always get back to a working state.

On github.com: **New repository** → name it (for example `coms3011a-exam`) → leave it **empty** → Create. Then:

```
cd ~/exam-app
git init
git add .
git commit -m "chore: scaffold project with plan"
git branch -M main
git remote add origin https://github.com/Suhai1/[repo-name].git
git push -u origin main
```

Username: `Suhai1`. Password: **paste your token** (Ctrl+Shift+V; nothing shows while you paste, and that's normal). Refresh GitHub: your files should be there.

STEP 4 — Check Qoder is set up properly (~5 min)

4a. Open the project

In Qoder: **File** → **Open Folder** → `exam-app` (the folder itself, not a file inside it). The file list on the left should show `AGENTS.md`, `src`, and so on.

4b. [QODER: Ask, Auto] Check it reads your rules

Why: if Qoder isn't reading `AGENTS.md`, it will use `@latest` packages and break Node 18, or delete rows. Thirty seconds now saves hours.

```
Without changing anything, answer briefly:
1. Which Node version and which pinned versions of next, tailwindcss, better-sqlite3 and vitest
must this project use? (from AGENTS.md)
2. List the database rules from AGENTS.md section 3 in one line each.
3. What are the project-specific rules in AGENTS.md section 8?
4. How many tasks are in PLAN.md, and what is Task 1?
5. Which files are in the project root?
```

It passed if: it says **Node 18**, **Tailwind 4.1.18**, **better-sqlite3 11.10.0**, and **Vitest 3.2.4**, mentions **never delete rows / archive**, and correctly lists your plan's tasks.

If it doesn't mention them: type `@AGENTS.md` and ask again. If that works, **start every prompt with** `@AGENTS.md` (the prompts below already do).

4c. Know your controls

Want to…	Do this
Fresh chat	**+** button in the chat panel (do this for every task)
Attach a file	type `@` then the file name
Accept / reject all changes	Ctrl+Enter / Ctrl+Backspace
Terminal	Ctrl+Backtick (the key left of 1)
Run the app	`npm run dev` in a terminal tab, then open http://localhost:3000

STEP 5 — The database foundation (~09:00, 20–30 min)

Why first: everything stores and reads data. If the database layer is solid and tested, every feature after it is just "a page that calls a function". This is the part you were worried about, so we make it small, check it, and lock it in with tests.

5a. [QODER: Agent, Auto, new chat] DB-1: build the data layer

```
@AGENTS.md @PLAN.md
Set up the database layer only. No pages, no API routes, no styling yet. Go step by step.
1. src/lib/migrations.ts: put the CREATE TABLE and CREATE INDEX statements from PLAN.md section 4
in as migration 1, using this pattern: a MIGRATIONS string array, and a migrate(db) function that
reads PRAGMA user_version and runs each pending migration inside BEGIN/COMMIT (ROLLBACK on error),
then sets user_version. Every table has id INTEGER PRIMARY KEY, created_at TEXT NOT NULL,
updated_at TEXT NOT NULL; archived_at TEXT for anything the user can "delete"; CHECK constraints
for fixed lists; foreign keys for relationships; join tables for many-to-many.
2. src/lib/db.ts: open SQLite with better-sqlite3 at process.env.DB_PATH or data/app.db (create
the data folder if missing, skip that for ":memory:"), set journal_mode = WAL and foreign_keys =
ON, run migrate(), and reuse one connection through globalThis so hot reload doesn't open new
ones.
3. src/lib/[main thing, e.g. tasks].ts: typed functions create, getById, listActive, update,
archive (and restore if the brief needs it). Validate input and throw clear Error messages. Use ?
placeholders only. Never DELETE. Set updated_at on every update.
4. vitest.config.ts with the "@" alias pointing to src, and src/tests/[main thing].test.ts with
tests for: create then read back; update changes the value and updated_at; archive hides it from
listActive but getById still finds it; empty or invalid input throws; an invalid value for a
CHECK-constrained column is rejected.
5. Run npm test and show me the full output.
```

Insert: your main data type from PLAN.md (for example `tasks`).

Check it (TERMINAL):

```
npm test
```

All tests should be green. Compare `src/lib/db.ts` with the reference in `pack/STACK-A-NEXTJS.md`; it should look about the same.

Commit: `git add . && git commit -m "feat(db): add database layer with migrations and tests"`

5b. If better-sqlite3 won't install: DB-FIX

Why: it downloads a pre-built file from GitHub that the firewall might block. sql.js does the same job using only npm.

[TERMINAL]

```
npm uninstall better-sqlite3 @types/better-sqlite3
npm install -E sql.js@1.15.2
npm install -E -D @types/sql.js
cp ~/pack/STACK-A-NEXTJS.md ~/exam-app/
```

[QODER: Agent]

```
@AGENTS.md @STACK-A-NEXTJS.md
better-sqlite3 cannot be installed on this machine. Switch to sql.js:
1. Replace src/lib/db.ts with the "Option 2: sql.js" db.ts code from STACK-A-NEXTJS.md exactly.
2. Set next.config.ts to the version shown there (serverExternalPackages: ["sql.js"]).
3. Make every function in src/lib async and start each with `const db = await getDb();`. Update
API routes and tests to await them.
4. Do not use db.pragma or db.transaction. Use db.exec("BEGIN") / db.exec("COMMIT") /
db.exec("ROLLBACK").
5. Update AGENTS.md section 1 to say we use sql.js via getDb().
Run npm test and npm run build.
```

Afterwards, delete the copy: `rm ~/exam-app/STACK-A-NEXTJS.md`.

5c. Prove the data really saves (do this after your first feature has a page)

Why: "survives a restart" is a classic marking check, and the demonstrator may test it.

1. [BROWSER] Create something.
2. [TERMINAL] See it in the database file:

```
python3 -c "import sqlite3; c=sqlite3.connect('data/app.db'); print([r[0] for r in
c.execute(\"select name from sqlite_master where type='table'\")])"
python3 -c "import sqlite3; c=sqlite3.connect('data/app.db'); [print(r) for r in c.execute('select
* from [table] order by id desc limit 5')]"
```

(**Insert:** your table name.)

3. Stop the app (**Ctrl+C**), run `npm run dev` again, refresh the browser. It must still be there.
4. Archive it in the app, then run the second python line again. **The row must still exist** with `archived_at` filled in.

STEP 6 — The look (~09:30, 30 min)

6a. [QODER: Agent, Auto, new chat] D1: visual foundation

Why: setting colours, fonts and the page frame **once** means every feature after this automatically looks consistent and not AI-generic.

```
@DESIGN.md @AGENTS.md @PLAN.md
Set up the visual foundation only. No product features yet.
1. Replace src/app/globals.css with the CSS in DESIGN.md §2.5 exactly (Forest Modern tokens,
@theme inline mapping, base styles, focus ring, reduced motion). Keep the @import "tailwindcss"
line first.
2. Remove any next/font/google imports (Geist) from src/app/layout.tsx; fonts come from the system
stacks in §3.
3. Build the app shell following §A (layout signature) and §6: numbered text sidebar (01, 02, ...)
with the sections from PLAN.md, compact top bar, page header component with the instrument index
eyebrow and registration rule.
4. Make the home page show the page header and an empty state (§7.12).
Run npm run build and tell me the result. Do not add features, libraries or new colours.
```

Check it: [BROWSER] warm off-white background, sidebar with numbered text links (01, 02…), a small "01 / OVERVIEW"-style label above the title and a dark line under it.

Commit: `git add . && git commit -m "style(ui): add forest modern foundation and shell"`

6b. [QODER: Agent, Auto, new chat] D2: UI kit

Why: a small set of ready-made buttons, fields, tables and empty states. Every feature reuses them, so features are faster to build and look the same.

```
@DESIGN.md @AGENTS.md
Create a small reusable UI kit in src/components/ui/ following DESIGN.md §A and §7 exactly. Use
only the tokens from §2.5 (Tailwind classes like bg-surface, text-ink-muted, border-line,
bg-accent). No new colours, no shadows except where §5.3 allows.
Components (typed props, keyboard accessible, all states from §7):
- Button (primary, secondary, ghost, danger; sizes sm/md; loading state that keeps width)
- Field (label + input/select/textarea + helper + error linked with aria-describedby)
- Badge (success, warning, danger, info, neutral; always shows text)
- Section (index label like "02 / ACTIVITY", title, 1px rule, children; NO card box)
- Panel (bordered surface, only for tables, forms and charts)
- DataTable (ledger style from §A.5: header rule, mono IDs, right-aligned numbers, sortable
headers, empty and loading states)
- Readout and ReadoutStrip (§A.3)
- EmptyState, ErrorBanner, Skeleton
Then show them all on a temporary page at /kit so I can check them. Run npm run build.
```

Check it: [BROWSER] http://localhost:3000/kit and look at each component.

Commit: `git add . && git commit -m "feat(ui): add ui kit"`

Tick the foundation task(s) in PLAN.md.

STEP 7 — Build features: the loop (~10:00 to 16:00)

Why this loop: one small task per **fresh** chat keeps the AI focused (long chats make it worse). Checking and committing after each task means a mistake can never cost you more than one task.

For EVERY task in PLAN.md:

1. [QODER: Agent, Auto] New chat → paste the Task Prompt + the Design line:

```
@PLAN.md @AGENTS.md @DESIGN.md
Do Task [N] from PLAN.md, and only that task.

1. First, restate the task's Goal, Rules to respect and Done-when in 3 lines, and list the files
you'll change. Wait for my OK.
2. Then implement it. Logic and SQL in src/lib, thin routes/pages, UI following DESIGN.md.
3. Add the tests listed for this task. Run `npm run build` and `npm test` and show me the results
honestly.
4. Tell me exactly how to check the Done-when myself (what to click / what to run).
5. When I confirm it works, tick Task [N] in PLAN.md as [x].
Do not start the next task. Do not change unrelated code. Do not install packages without asking
(Node 18, pinned versions).

Design: use only components from src/components/ui and tokens from DESIGN.md. Follow §A (no card
grids, readout strip for numbers, ledger tables, sections with index labels). Every list needs
empty, loading and error states. No new colours, gradients, shadows or rounded cards.
```

Insert: the task number in both `[N]` places.

2. Read its 3-line summary. Wrong? Correct it. Right? Reply `OK`.

3. While it works: when it asks to run a command, **read it**. Approve `npm test`, `npm run build` and `npm install` of something you agreed to. **Reject** `npm update`, `npm audit fix --force`, anything `@latest`, and `rm -rf`.

4. Before accepting: glance at the changed files. Did it touch files unrelated to the task? Is there a `DELETE FROM`, or a new status value the brief doesn't allow? If so, reject (Ctrl+Backspace) and say why.

5. [BROWSER] Check it yourself. Do the task's **"Done when"** check exactly. Never trust "it works!" until you've seen it.

6. [TERMINAL] Commit:

```
npm test
git add .
git commit -m "feat([area]): [what it does]"
```

7. Next task → new chat.

When a task needs a database change: DB-2

Why: you'll often discover you need another column or table. Doing it as a **new migration** keeps existing data and never breaks old code.

[QODER: Agent]

```
@AGENTS.md @src/lib/migrations.ts
I need [describe it, e.g. "a priority field on tasks (1-4, default 2)" or "a tags table linked to
tasks (many-to-many)"].
Add it as a NEW migration at the END of the MIGRATIONS array. Do not edit earlier migrations, and
do not drop or recreate tables. New columns use ALTER TABLE ... ADD COLUMN with a NOT NULL DEFAULT
(or nullable). New tables get id, created_at, updated_at, foreign keys and indexes; many-to-many
uses a join table.
Then update the TypeScript types and the lib functions that read or write it, and add a test. Run
npm test.
```

During development only: if the database gets into a weird state, it's fine to start fresh: stop the app, run `rm -f data/app.db*`, start it again (migrations rebuild the tables), then `npm run seed` if you have seed data.

When data doesn't save, doesn't show, or shows old values: DB-3

Why: the three usual causes are the lib function, the route, or Next.js caching an old page. This makes the AI check all of them instead of guessing.

[QODER: Agent]

```
@AGENTS.md
Data problem. I did: [steps]. I expected: [result]. I got: [what happened]. Error text: [paste or
"none"].
Before changing anything, check in this order and tell me which one is the root cause:
1. Does the page/form actually call the API route (check the fetch URL, method and body)?
2. Does the route call the right lib function with the right values and return errors as JSON with
a status code?
3. Does the lib function's SQL write/read the right table and columns (and filter archived_at
correctly)?
4. Is Next.js showing cached data? Pages and routes that read the database must be dynamic: export
const dynamic = "force-dynamic", fetch with cache: "no-store", and call router.refresh() after
saving from a client component.
5. Are the tests using :memory: while the app uses data/app.db (that's expected; don't change it)?
Then fix the root cause only, add a test that would have caught it, and run npm test.
```

When something else breaks: Fix prompt

```
Bug: [what happens]
Expected: [what should happen]
Steps to reproduce: [1, 2, 3]
Error message (if any): [paste it exactly]
Where I think it is: [file or "not sure"]

Find the root cause first and explain it in 2–3 sentences before changing anything.
Then make the smallest fix. Add a test that would have caught this. Run tests.
```

When it goes in circles (same fix failed twice): switch the model to Ultimate

```
Stop. We've tried this [N] times and it's not working.
Don't write code yet. Explain:
1. What you think the root cause is.
2. What assumption you've been making that might be wrong.
3. Two different approaches, and which you recommend.
```

Still stuck after that? [TERMINAL] `git stash`, open a new chat, and ask again using what you learned. `git stash` puts the broken changes aside; nothing is lost.

Every 3–4 tasks, or when PLAN.md says VERIFY: [QODER: Agent, new chat]

```
@PLAN.md @AGENTS.md
VERIFY checkpoint. Don't add features.
1. Run npm run build and npm test; report results.
2. List every completed task's Done-when check from PLAN.md so I can click through them all now.
3. Review the code changed since the last checkpoint against AGENTS.md section 3 (database rules)
and list any violations.
Fix only real breakages, one at a time, with my OK.
```

Also every 3–4 tasks: a fresh reviewer [QODER: Ask, new chat]

Why: the AI that built something is bad at spotting its own mistakes. A fresh chat with "examiner" instructions catches them.

```
You are a strict code reviewer and examiner. Review this project against @AGENTS.md and @BRIEF.md.
Do NOT change any code. Give me a list, most serious first:
1. Any rule in AGENTS.md that's violated (deleting rows, stored derived values, extra enum values,
N+1 queries, missing indexes, no cycle guards, drop-and-recreate migrations, duplicate libraries).
2. Any "done when" condition in the brief that the current code would FAIL in a live demo.
3. Bugs and unhandled edge cases (empty input, invalid input, dates/timezones, restart
persistence).
4. Tests that are fake (mocked DB, only check rendering).
For each: file, line/area, why it matters, suggested fix in one sentence.
```

Fix the top items one at a time with the Fix prompt.

STEP 8 — Every hour on the hour (set phone alarms 10:00 → 16:00)

Why: this catches silent breakage early, and your work is always safe on GitHub.

1. [TERMINAL] `git push`
2. [BROWSER] Click through the **whole app** like a user, start to finish.
3. [TERMINAL] `npm test`
4. Something broken? Fix it **before** the next task.
5. Behind schedule? [QODER: Ask]

```
@PLAN.md
It's [time]. Tasks [x, y] are done. I'm behind / [what changed].
Re-plan the remaining tasks to finish by 16:00. Keep all core tasks. Drop or shrink optional tasks
with the lowest marks ÷ effort, and tell me what you dropped. Update PLAN.md.
```

Lunch: push before you leave. When you're back, walk through the app before doing anything else.

STEP 9 — Realistic data (once the core works, ~13:30)

Why: an app with "Task 1, Task 2" looks unfinished. Realistic data makes every screen look real in the demo.

[QODER: Agent, new chat]

```
Create scripts/seed.ts and an npm script "seed" that fills the database with [~200] realistic
records for [describe domain].
Realistic names/titles (not "Task 1"), a spread of dates (past, today, future), all statuses, and
some relationships.
It must use the lib functions (not raw inserts that bypass rules) and be safe to run on an empty
database.
```

Insert: about 150–200 records, and a description of your app's data.

Check it: [TERMINAL] `rm -f data/app.db* && npm run seed && npm run dev`, then look at every screen.

STEP 10 — Make it look designed (~14:00, after the minimum passing version)

10a. [QODER: Ask, Auto, new chat] D4: find what looks generic

```
@DESIGN.md
You are a strict design reviewer. Do NOT change code. Go through every page in src/app and check
it against DESIGN.md §A ("the generic test") and §14 (forbidden defaults).
List every problem, worst first, with the file and a one-line fix:
- rows of 3+ equal boxes/cards, cards around every section, rounded big containers
- icon-only navigation, centred narrow layouts, missing index labels or registration rule
- colours not from §2.5 tokens, gradients, shadows, glow, purple/blue SaaS look
- missing empty/loading/error states, colour-only status, missing focus styles
- numbers not in mono/tabular, numbers left-aligned in tables
```

10b. [QODER: Agent, Auto, new chat] D5: fix it

```
@DESIGN.md @AGENTS.md
Fix these design problems, one page at a time, without changing any behaviour or data logic:
[paste D4's list here]
After each page, run npm run build. List what you changed.
```

Insert: paste D4's list where it says `[paste D4's list here]`.

Then: `npm test`, click through the app, and commit `style(ui): design pass`.

10c. Only if the core is done and it's before 15:30: D6, one wow detail

```
@DESIGN.md
Add ONE optional detail from DESIGN.md §15: [Live Result Tally | Focused Record Panel | Delta
Strip | Change Ledger].
It must use real app data (no fake numbers), use only existing UI kit components and tokens, and
must not change existing behaviour. Keep it under 20 minutes. Run npm run build and npm test.
```

Commit it **on its own**. If it breaks something, `git revert HEAD` undoes just that commit.

Any time a page looks like a template: D7 (Ultimate)

```
@DESIGN.md
This page (@[page file]) still looks like a default AI SaaS dashboard. Rework its LAYOUT (not its
colours or behaviour) to follow DESIGN.md §A strictly:
- replace any row of equal cards with a ReadoutStrip (one oversized primary number, secondary
readouts separated by vertical rules)
- remove card boxes around sections; use Section (index label + title + rule) on the canvas
- put tables in ledger style, full width
- use the 2fr/1fr main + inspection rail split on wide screens
Keep all existing functionality working. Show me before/after of what changed.
```

STEP 11 — Feature freeze and finish (16:00–17:00)

Why: a half-working feature in a live demo loses more marks than a missing one. From 16:00 you only fix and polish.

11a. 16:00 [QODER: Agent, new chat] Final checklist

```
Final pre-submission check. Do not add features.
1. Run the build and all tests; fix anything failing.
2. Write/update README.md: what the app does, features implemented, exact commands to install,
run, seed and test.
3. Check .gitignore excludes node_modules, .next, data/*.db, .env.
4. Remove debug code, console.logs and dead files.
5. List any feature that is half-done so I can decide whether to hide it.
```

11b. [QODER: Agent, new chat] D8: final visual check

```
@DESIGN.md
Final visual check, no new features:
1. Every page reachable from the sidebar renders without errors and has the page header with index label.
2. No dead buttons, placeholder links, "Lorem ipsum", or template leftovers (Next.js logo, default favicon, /kit in nav).
3. Every list has an empty state; every form shows errors.
4. Check at 1440px and at 390px wide: no horizontal page overflow.
Fix only what's broken. Run npm run build.
```

11c. 16:30 — Full walkthrough

1. [TERMINAL] `rm -f data/app.db* && npm run seed` (clean demo data), then `npm run build` and `npm test`.
2. [TERMINAL] `npm run dev`, then [BROWSER] do **every "Done when" check** in PLAN.md, in order.
3. Do the **restart test** from Step 5c once more.

11d. 16:40 — Final push and submit

```
git add .
git commit -m "chore: final submission"
git push
```

- [BROWSER] Open your repo on **github.com**. Check the latest commit message and time are there.
- **Submit** exactly as the exam instructions say (probably your repo link on Moodle). Read the instructions at 08:00 so you know.
- Have the app **running** (`npm run dev`) if the demonstrator marks live at your PC.

11e. Before you leave the PC

```
git credential-cache exit
```

This logs your token out. Then sign out of Qoder. After the exam, delete the token on GitHub.

Emergency table

Problem	Do this
`npm run build` fails with **tailwindcss-oxide**	`npm install -E -D tailwindcss@4.1.18 @tailwindcss/postcss@4.1.18`
better-sqlite3 won't install	Step 5b (DB-FIX)
`File is not defined` / `toSorted is not a function`	Node 18 limits. See `pack/STACK-A-NEXTJS.md` → "Node 18 gotchas"
Data not saving / old data showing	DB-3
Need a new column/table	DB-2
Database in a weird state (development only)	Stop the app, `rm -f data/app.db*`, start it, `npm run seed`
AI going in circles	Loop breaker on Ultimate, then `git stash` and a new chat
AI changed lots of unrelated files	Reject all, then "Only change X in @file"
Broke something that worked an hour ago	`git log --oneline`, then `git revert [hash]`
Push asks for a password	Username `Suhai1`, password = your token
Qoder down	Keep going in the terminal; use Gemini in the browser for questions
Running out of time	Re-plan prompt (Step 8), drop optional tasks, protect the core